Emergency Logistics & Resource Allocation Network

Data Structures Project Report - Graph (adjacency list) and Min-Heap, implemented in Java

Student name	_____
Roll number	_____
Class / Division	_____
Subject	Data Structures
Faculty guide	_____
Institution	_____
Date	_____

1. Abstract

In a disaster, relief centers compete for limited stock stored in several warehouses. Supplies travel over roads that have different travel times and limited capacity, and roads can fail. This project decides **what to send, from which warehouse, along which route** and re-decides when roads fail, fill up, or stock changes.

Exactly two data structures carry the whole system, both written from scratch: a **Graph** (adjacency list) that models the roads, and a **Min-Heap** that is used for two unrelated jobs: ordering requests by urgency, and driving Dijkstra's shortest-path search.

2. Problem statement and constraints

Constraint in the problem	How the system handles it
Roads have different travel times	Edge weight = minutes; Dijkstra finds the fastest route.
Roads have limited capacity	Each road has a capacity per wave; a shipment is limited by the weakest road on its route, and full roads are skipped by later searches.
Roads can become unavailable	A road has a blocked flag; blocked edges are skipped, so routes change automatically.
Warehouses have limited inventory	Stock is kept per warehouse and resource; a shipment can never exceed it. Leftover demand goes to the next-nearest warehouse.
Several centers compete for the same stock	An urgency min-heap decides the order: higher severity first, then more critical resource, then earlier request.

3. How the system decides who gets what

All requests that arrive together are pushed into one min-heap. A request is ordered by **(1)** center severity, highest first (hospital = 5), **(2)** resource criticality (MEDICINE, then WATER, then FOOD), **(3)** arrival order. The most urgent request is always the heap root.

```
dispatchWave():
    reset every road's free capacity                  // new convoy cycle
    while urgency heap is not empty:
        r = heap.pop()                                // most urgent request
        s = Dijkstra(from r.center)                   // graph + MinHeap; skips blocked/full roads
              stops at the nearest warehouse that has stock of r.resource
        if none found: r stays pending for the next wave; continue
        take = min(r.need, warehouse stock, weakest road capacity on route)
        stock -= take ; every road on route: free capacity -= take ; r.need -= take
        record shipment
        if r.need > 0: heap.push(r)                   // remainder re-enters the heap
```

Because the remainder is pushed back, a large order is automatically split across several warehouses and roads. Running Dijkstra again after every shipment is needed because each shipment consumes road capacity and can change the best route.

4. Why these two data structures

4.1 Graph (adjacency list)

A road network has loops and many-to-many connections, so alternate routes exist. When a road fails we disable one edge and the other paths stay usable.

Alternative	Why not
Tree	Exactly one path between two nodes: a failed road disconnects part of the map, no rerouting possible.
Array / list of places	Stores places but not the connections between them.
Adjacency matrix	V x V memory; road maps are sparse (2-5 roads per junction) so most cells would be empty, and listing neighbours costs O(V).
Adjacency list	Memory O(V+E); the roads leaving a node are read directly from its list, exactly what Dijkstra needs. One Road object is shared by both end points, so blocking it or using its capacity affects both directions.

4.2 Min-Heap (priority queue)

The system needs one operation again and again: *give me the best item next while new items keep arriving*. That is needed in two places that have nothing to do with each other.

Job	What is "best"	Comparator
Request queue (ReliefSystem)	Most urgent request	severity (high first), resource criticality, arrival order
Dijkstra frontier (RoadGraph)	Closest unfinished location	distance (low first)

Alternative	Insert	Take best	Problem
Unsorted array	O(1)	O(n)	Scans everything each time; Dijkstra becomes O(V squared).
Sorted array / list	O(n)	O(1)	Every new request or distance forces shifting.
Plain queue (FIFO)	O(1)	O(1)	Serves by arrival, so a low-priority request that came first beats the hospital.
Stack (LIFO)	O(1)	O(1)	Serves the newest first; meaningless for priority.
Min-Heap	O(log n)	O(log n)	None for this use; stored in a plain array.

One generic `MinHeap<T>` class serves both jobs; only the `Comparator` changes. *Honest note:* on a tiny map any structure can be made to work; these are chosen for efficiency and natural fit.

5. Code structure

File	Responsibility
`backend/MinHeap.java`	Generic min-heap: push, pop, copy. Counters for pushes/pops are shown in the UI.
`backend/RoadGraph.java`	Graph as adjacency list (Road, linked cells), blocking, capacity, Dijkstra using MinHeap, path rebuilding.
`backend/ReliefSystem.java`	Urgency heap, requests, stock, dispatch wave: the decision logic.
`backend/Main.java`	Small JDK web server; prints the system state as JSON.
`frontend/index.html`	One page: map, heap array view, stock, shipments, decision log, adjacency list.

Only `java.util.Comparator` and `ArrayList` (for plain log lines in the UI) are borrowed from the JDK. The graph and the heap are fully hand written.

5.1 Heap operations

Operation	What it does	Time
push	Add at the end of the array; swap upward while smaller than the parent.	O(log n)
pop	Take the root; move the last element to the root; swap downward with the smaller child until the rule holds.	O(log n)
copy	Duplicate the array so the UI can show the serving order without changing the real queue.	O(n)

5.2 Graph storage

```
head[J3] -> [J3-C4] -> [J3-C2] -> [J3-C1] -> [J2-J3] -> [J1-J3] -> [W3-J3] -> null
(the Road J1-J3 object is the SAME object in head[J1]'s list: block it once, closed both ways)
```

6. Complexity

V = locations, E = roads, R = requests, W = warehouses.

Operation	Time
One Dijkstra run (stops early at the nearest warehouse with stock)	O((V+E) log V)
One shipment	one Dijkstra run, then O(path length) to update stock and road capacity
Handle one request fully	O(k (V+E) log V) where k = number of shipments needed (usually 1-3)
Urgency heap: add or serve a request	O(log R)
Block / reopen a road	O(1) once found (name lookup scans the 16 roads)
Memory	O(V + E + R)

7. Testing (real runs on the sample map)

Sample map: 3 warehouses, 3 junctions, 4 relief centers (C2 hospital severity 5, C4 severity 4, C1 severity 3, C3 severity 2), 16 roads with capacities 20-40 per wave. Seven requests totalling 275 units arrive together.

#	Case	Observed result
1	7 simultaneous requests, no shipping yet	Serve order from the heap: C2 MEDICINE 40, C2 WATER 50, C4 MEDICINE 25, C4 WATER 30, C1 WATER 40, C1 FOOD 30, C3 FOOD 60
2	Wave 1: hospital medicine 40	30 from W3 via W3-J3-C2 (road J3-C2 capacity 30 is full), remaining 10 from W2 via W2-C4-C3-C2 (40 min). Remainder was re-pushed into the heap.
3	Wave 1 totals	9 shipments, 135 units; 140 units still pending because roads into C1, C3 were full
4	Waves 2 and 3	Wave 2 delivers 90 more units (225 total); wave 3 clears the rest: pending = 0
5	Block W3-J3 and J3-C2; C2 needs 30 MEDICINE	Served from W2: 25 via W2-C4-C3-C2 (limited by road capacity 25) and 5 via W2-J2-J3-C1-C2. Total 30

#	Case	Observed result
6	Block all three roads into C2; C2 needs 10 FOOD	UNMET: no reachable warehouse; request stays pending
7	Request for a non-center (W1)	Error: W1 is not a relief center
8	Heap counters after wave 1	Urgency heap: 19 pushes / 14 pops; Dijkstra: 14 runs, 67 heap pops

8. Limitations and future work

- Strict priority can starve low-priority centers: in wave 1 the hospital's reroutes used up road capacity that C1 and C3 also needed. A fairness cap per center would help.
- The greedy method is not globally optimal. The exact problem is a min-cost multi-commodity flow, a natural extension on the same graph.
- Dijkstra uses lazy deletion (duplicate heap entries). An indexed heap with decrease-key would keep the heap at size V.
- Time is simplified to waves with refilled capacity; there are no truck fleets or repair schedules. The map is fixed in code.

9. Likely viva questions

Q. Why not a queue for requests?

A. A queue serves by arrival time, so a low-priority request that came first would beat the hospital. A heap serves by priority.

Q. Why use the same heap for requests and for Dijkstra?

A. Both need 'insert, then give me the best next' in O(log n). Only the meaning of best differs, so one class with a Comparator serves both.

Q. How do you decide who gets what when stock is short?

A. Requests come out of the heap by severity, then resource criticality, then arrival. Each one takes from the nearest warehouse, limited by stock and road capacity; whatever remains is pushed back.

Q. What happens when a road fails?

A. Its blocked flag is set. Dijkstra skips it, so the next search finds another route. If none exists the request stays pending.

Q. Why re-run Dijkstra after every shipment?

A. A shipment uses road capacity, which can change the best route for the next one.

Q. Is the result optimal?

A. No. It is a greedy priority method. It guarantees the most urgent needs are served first, not that total delivery time is minimal.

Q. Why adjacency list and not matrix?

A. Road maps are sparse; the list needs O(V+E) memory and gives a node's roads directly.

10. Conclusion

Two data structures, used for real work, handle every constraint of the problem: the graph represents the roads and their failures and capacities, and the min-heap decides both who is served first and which location to explore next.